

02 — Multi-Version Concurrency Control (MVCC)

Table of Contents

1. [Learning Objectives](#)
 2. [Why MVCC Exists](#)
 3. [Core Concepts: xmin, xmax, xip](#)
 4. [Tuple Versions and the Tuple Chain](#)
 5. [Transaction Snapshots](#)
 6. [Snapshot Visibility Rules](#)
 7. [How Reads Never Block Writes](#)
 8. [The Commit Log \(pg_xact\)](#)
 9. [MVCC and UPDATE / DELETE](#)
 10. [MVCC and Dead Tuples](#)
 11. [Observing MVCC with System Columns](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions and Answers](#)
 16. [Exercises and Solutions](#)
 17. [Production Troubleshooting Scenarios](#)
 18. [Cross-References](#)
-

Learning Objectives

After studying this file you will be able to:

- Explain the fundamental purpose of MVCC and how it differs from lock-based concurrency
 - Describe the system columns xmin, xmax, cmin, cmax, ctid and their roles
 - Trace a tuple version chain and determine which version is visible to a given snapshot
 - Explain transaction snapshot semantics (xmin, xmax, xip_list)
 - Explain why reads in PostgreSQL never block writes and vice versa
 - Describe how dead tuples arise and why VACUUM is needed
 - Query system columns to observe MVCC in action
-

Why MVCC Exists

Traditional databases use a **two-phase locking (2PL)** model: a reader must wait for writers, and writers must wait for readers. This causes severe contention on hot rows.

PostgreSQL's **Multi-Version Concurrency Control (MVCC)** solves this by keeping **multiple versions** of each row simultaneously. Each transaction sees a consistent snapshot of the database **as of the moment the snapshot was taken**, regardless of what other transactions are doing concurrently.

Lock-based model:

Reader ----WAIT----> Writer holds lock ----WAIT----> Reader

MVCC model:

Reader ---reads old version---> no wait

Writer ---writes new version--> no wait for reader

The trade-off: old versions accumulate as "dead tuples" and must eventually be reclaimed by VACUUM.

Core Concepts: xmin, xmax, xip

Every heap tuple (row version) stores four hidden system columns:

Column	Type	Meaning
xmin	xid	XID of the transaction that inserted this tuple version
xmax	xid	XID of the transaction that deleted or updated this tuple (0 = live)
cmin	cid	Command ID within the inserting transaction
cmax	cid	Command ID within the deleting transaction
ctid	tid	Physical location (page, offset) of this tuple version

A transaction's **snapshot** captures three values at the moment it is taken:

- `xmin` — the oldest XID still active at snapshot time (all XIDs < xmin are committed)
- `xmax` — the next XID to be assigned (all XIDs >= xmax haven't started yet)
- `xip_list` — list of XIDs in the range [xmin, xmax) that were active (in-progress) at snapshot time

Tuple Versions and the Tuple Chain

When a row is updated, PostgreSQL does NOT modify the tuple in place. Instead:

- The old tuple version gets its `xmax` set to the updating transaction's XID.
- A **new tuple version** is inserted with `xmin` = updating XID.
- The new version's `ctid` in the old version points to the new physical location.

This creates a **version chain** of tuples:

```
Initial INSERT by Txn 100:
+-----+
| xmin=100 | xmax=0    | ctid=(0,1) | data='A' | <-- current version
+-----+

UPDATE by Txn 200 (changes 'A' -> 'B'):
+-----+
| xmin=100 | xmax=200 | ctid=(0,2) | data='A' | <-- dead after Txn 200 commits
+-----+
|
| v (ctid forward pointer)
+-----+
| xmin=200 | xmax=0    | ctid=(0,2) | data='B' | <-- current version
+-----+

UPDATE by Txn 300 (changes 'B' -> 'C'):
+-----+
| xmin=100 | xmax=200 | ctid=(0,2) | data='A' | <-- dead
+-----+
```

```

      |
      v
+-----+
| xmin=200 | xmax=300 | ctid=(0,3) | data='B' | <-- dead after Txn 300 commits
+-----+
      |
      v
+-----+
| xmin=300 | xmax=0    | ctid=(0,3) | data='C' | <-- current version
+-----+

```

Key insight: every UPDATE produces a new physical row. The old version is NOT removed until VACUUM runs.

Transaction Snapshots

A snapshot is captured when:

- In `READ COMMITTED` (default): at the start of **each statement**
- In `REPEATABLE READ` / `SERIALIZABLE` : at the start of the **first statement** of the transaction

Snapshot Structure

```
Snapshot = (xmin=500, xmax=510, xip=[502, 507])
```

Interpretation:

- All XIDs < 500 are committed (or aborted) – definitely visible or not
- XIDs 500..509 are the "interesting" range
- XIDs 502 and 507 were still in-progress when snapshot was taken
- XIDs 500, 501, 503, 504, 505, 506, 508, 509 committed before snapshot
- All XIDs >= 510 had not started yet – not visible

```

-- See current snapshot
SELECT txid_current_snapshot();
-- Returns: 500:510:502,507
-- Format:  xmin:xmax:xip_list

```

Snapshot Visibility Rules

A tuple version (with `xmin` T1 and `xmax` T2) is **visible** to a snapshot S if and only if:

1. T1 (creator) is COMMITTED w.r.t. snapshot S:
 - T1 < S.xmin (committed before snapshot was taken), OR
 - T1 in [S.xmin, S.xmax) AND T1 NOT in S.xip AND T1 committed

AND

2. T2 (deleter) is NOT committed w.r.t. snapshot S:
 - T2 = 0 (tuple not deleted), OR
 - T2 >= S.xmax (started after snapshot), OR

- T2 in S.xip (still in-progress at snapshot time), OR
- T2 aborted

Worked Example

Snapshot S = (xmin=500, xmax=510, xip=[505])

Tuple A: xmin=490, xmax=0 --> VISIBLE (490<500 committed, not deleted)
 Tuple B: xmin=490, xmax=503 --> NOT VISIBLE (503 committed before snapshot, deleted it)
 Tuple C: xmin=503, xmax=0 --> VISIBLE (503 committed before snapshot, not deleted)
 Tuple D: xmin=505, xmax=0 --> NOT VISIBLE (505 in xip = still in-progress)
 Tuple E: xmin=511, xmax=0 --> NOT VISIBLE (511 >= 510, future transaction)
 Tuple F: xmin=490, xmax=505 --> VISIBLE (505 is in xip = deleter still in-progress)

How Reads Never Block Writes

This is the core value proposition of MVCC:

1. A **reading transaction** takes a snapshot at statement start. It reads only tuple versions visible to that snapshot. It does NOT acquire any row-level read locks.
2. A **writing transaction** inserts new tuple versions or marks old ones with xmax. It does NOT need to check what readers are doing.
3. Therefore, **readers and writers proceed concurrently without blocking each other.**

Time -->

Txn A (reader, snapshot at T=100):

|--SELECT sees version V1 of row----->|

Txn B (writer):

|--UPDATE creates V2 of row, xmax V1-->|

|--COMMIT----->|

Txn A still sees V1 because its snapshot was taken before Txn B committed.
 No locks were exchanged between A and B.

The only cases where reads block:

- `SELECT ... FOR UPDATE/SHARE` explicitly requests a row lock
- DDL (schema changes) acquire table-level locks

The Commit Log (pg_xact)

PostgreSQL maintains a **commit log** (`$PGDATA/pg_xact/`) that records the status of every transaction:

Status	Meaning
IN_PROGRESS	Transaction is still running

COMMITTED	Transaction committed
ABORTED	Transaction was rolled back
SUB_COMMITTED	Subtransaction committed (rare)

When checking tuple visibility, PostgreSQL looks up `xmin` and `xmax` in `pg_xact` to determine if those transactions committed or aborted. The commit log is a simple bitmap, very fast to read. Frequently-accessed entries are cached in shared memory (`clog` buffer cache).

```
-- PostgreSQL 10+ renamed pg_clog to pg_xact
-- View via:
SELECT * FROM pg_xact_status(500::xid); -- not directly exposed; use txid_status()
SELECT txid_status(500); -- 'committed', 'aborted', 'in progress', or NULL
```

Hint Bits

To avoid repeated `pg_xact` lookups, PostgreSQL sets **hint bits** directly on tuple headers once the transaction status is known:

- `HEAP_XMIN_COMMITTED` — `xmin`'s transaction committed
- `HEAP_XMIN_INVALID` — `xmin`'s transaction aborted
- `HEAP_XMAX_COMMITTED` — `xmax`'s transaction committed
- `HEAP_XMAX_INVALID` — `xmax`'s transaction aborted

Setting a hint bit is a "dirty" write to the page. It is WAL-logged as an FPI (Full Page Image) on the first write after a checkpoint to avoid torn pages.

MVCC and UPDATE / DELETE

UPDATE Internals

```
UPDATE employees SET salary = 90000 WHERE id = 42;
```

Internally:

1. Find the current tuple version for `id=42` (`ctid = (3, 7)`)
2. Lock the tuple (set `xmax = current_xid` tentatively)
3. Write a new tuple version at `ctid = (3, 15)` with `xmin = current_xid`, new salary
4. Update the old tuple's `xmax = current_xid`
5. At COMMIT: `pg_xact` records `current_xid` as committed

DELETE Internals

```
DELETE FROM employees WHERE id = 42;
```

1. Find the current tuple version
2. Set `xmax = current_xid`
3. At COMMIT: `pg_xact` records `current_xid` as committed
4. No new tuple version is created — the tuple simply becomes invisible to future snapshots

The physical space is NOT reclaimed until VACUUM.

INSERT Internals

```
INSERT INTO employees (id, name, salary) VALUES (99, 'Alice', 80000);
```

1. Write a new tuple with xmin = current_xid, xmax = 0
2. At COMMIT: pg_xact records current_xid as committed

MVCC and Dead Tuples

After UPDATE or DELETE commits, the old tuple version remains on the heap page. It is called a **dead tuple** because no future snapshot can see it (xmax is committed). Dead tuples:

- Waste disk space
- Slow down sequential scans (must skip dead tuples)
- Slow down index scans (index entries still point to dead heap tuples)
- Cause **table bloat** and **index bloat**

VACUUM removes dead tuples that are no longer needed by any open snapshot.

Before VACUUM:

Page (8KB):

```
+-----+-----+-----+-----+
| V1 dead| V2 live| V3 dead| V4 live| (empty)|
+-----+-----+-----+-----+
```

After VACUUM:

```
+-----+-----+-----+
| V2 live| V4 live| (free) |
+-----+-----+-----+
(V1 and V3 removed, space reclaimed)
```

Observing MVCC with System Columns

```
-- Create demo table
CREATE TABLE mvcc_demo (id int, val text);

INSERT INTO mvcc_demo VALUES (1, 'initial');

-- See xmin, xmax, ctid
SELECT xmin, xmax, ctid, id, val FROM mvcc_demo;
/*
  xmin | xmax | ctid  | id |  val
-----+-----+-----+----+----
   501 |    0 | (0,1) |  1 | initial
*/

-- Update and observe
```

```

BEGIN;
SELECT txid_current(); -- say, returns 502
UPDATE mvcc_demo SET val = 'updated' WHERE id = 1;
SELECT xmin, xmax, ctid, id, val FROM mvcc_demo;
-- Within the same txn, you see the NEW version:
/*
  xmin | xmax | ctid  | id |   val
  -----+-----+-----+----+-----
    502 |    0 | (0,2) |  1 | updated
*/
COMMIT;

-- After commit, from another session that started BEFORE the commit:
-- Still sees xmin=501, val='initial' (READ COMMITTED would see new value)

```

Seeing Dead Tuples

```

-- Count dead tuples
SELECT relname, n_dead_tup, n_live_tup, last_vacuum
FROM pg_stat_user_tables
WHERE relname = 'mvcc_demo';

-- Use pageinspect extension
CREATE EXTENSION IF NOT EXISTS pageinspect;

SELECT lp, lp_flags, t_xmin, t_xmax, t_ctid, t_data
FROM heap_page_items(get_raw_page('mvcc_demo', 0));

```

Common Mistakes

1. Expecting Immediate Disk Reclamation After DELETE

```

DELETE FROM large_table WHERE created_at < '2020-01-01';
-- Table size has NOT shrunk! Dead tuples remain until VACUUM/VACUUM FULL.

```

2. Long Transactions Preventing Dead Tuple Cleanup

A snapshot from a long-running transaction holds back VACUUM. VACUUM cannot remove any dead tuple that might be visible to that snapshot.

```

-- Find the oldest snapshot holding VACUUM back
SELECT pid, now() - xact_start, query
FROM pg_stat_activity
WHERE xact_start = (SELECT min(xact_start) FROM pg_stat_activity WHERE xact_start IS NOT NULL);

```

3. Thinking xmax=0 Always Means "Live"

`xmax=0` means no transaction has tried to delete/update this tuple. But you also need `xmin` to be committed and visible. An uncommitted INSERT has `xmin = in-progress XID`.

4. Ignoring Table Bloat

Heavy UPDATE workloads produce many dead tuples. Without proper autovacuum tuning, tables balloon to 10x their logical size.

Best Practices

1. **Run VACUUM regularly** — let autovacuum handle it, but tune `autovacuum_vacuum_scale_factor` for write-heavy tables.
2. **Monitor dead tuples** — alert when `n_dead_tup / n_live_tup > 0.2` for large tables.
3. **Keep transactions short** — long transactions age snapshots and block cleanup.
4. **Use `pageinspect`** in development to verify your understanding of tuple versions.
5. **Avoid VACUUM FULL during peak hours** — it rewrites the whole table with an exclusive lock.
6. **Set `fillfactor < 100`** on write-heavy tables so UPDATES can use HOT (Heap-Only Tuple) optimization.

Performance Considerations

HOT Updates

If an UPDATE does not change any indexed column and the new tuple fits on the same page, PostgreSQL performs a **Heap-Only Tuple (HOT)** update:

- No new index entry is needed
- The old tuple's `xmax` points to the new tuple on the same page
- Reduces index bloat significantly

```
-- Check HOT update rate
SELECT relname, n_tup_upd, n_tup_hot_upd,
       round(100.0 * n_tup_hot_upd / nullif(n_tup_upd, 0), 1) AS hot_pct
FROM pg_stat_user_tables
WHERE n_tup_upd > 0;
```

Snapshot Age and Performance

Taking a snapshot requires checking the `proccarray` (all active transaction IDs). This is $O(\text{number of active transactions})$ and is protected by a lightweight lock. High connection counts slow snapshot acquisition.

Vacuum Cost

VACUUM competes with normal I/O. PostgreSQL's cost-based vacuum delay (`vacuum_cost_delay` , `vacuum_cost_limit`) throttles VACUUM to avoid starving production queries.

Interview Questions and Answers

Q1. What does MVCC stand for and what problem does it solve?

A: Multi-Version Concurrency Control. It solves the reader-writer blocking problem in lock-based systems by keeping multiple versions of rows and giving each transaction a consistent snapshot without taking row-level read locks.

Q2. What are xmin and xmax in a PostgreSQL tuple?

A: `xmin` is the XID of the transaction that created (inserted) this tuple version. `xmax` is the XID of the transaction that deleted or updated (superseded) this tuple version. `xmax=0` means the tuple has not been deleted or updated.

Q3. Why does an UPDATE create two tuples instead of modifying the existing one?

A: To support MVCC. Concurrent readers may have snapshots that need to see the old version. Modifying in place would destroy their consistent view. The old version remains until no snapshot can see it anymore (VACUUM removes it then).

Q4. What is a transaction snapshot and what are its three components?

A: A snapshot captures the state of all active transactions at a point in time. Its components are:

- `xmin` : the oldest active XID — all XIDs before this are definitely committed or aborted
- `xmax` : the next XID to be assigned — all XIDs $\geq$ this are future transactions
- `xip_list` : XIDs in the range `[xmin, xmax)` that were still in-progress at snapshot time

Q5. Why can reads never block writes in PostgreSQL (under normal circumstances)?

A: Readers only take a snapshot (no row locks). Writers insert new tuple versions (no need to notify readers). They operate on different tuple versions. The only exception is `SELECT ... FOR UPDATE/SHARE`.

Q6. What is a dead tuple and how does it arise?

A: A dead tuple is a tuple version whose `xmax` is a committed transaction — meaning it was deleted or superseded by an UPDATE that committed. No future snapshot can see it, but it remains on the heap page occupying space until VACUUM.

Q7. What are hint bits and why do they exist?

A: Hint bits are flags stored in the tuple header (`HEAP_XMIN_COMMITTED` , `HEAP_XMAX_COMMITTED` , etc.) that cache the result of a `pg_xact` lookup. Without hint bits, every visibility check would require reading `pg_xact`. Setting a hint bit is idempotent and results in a "dirty" write (WAL-protected after checkpoint).

Q8. How does PostgreSQL handle a DELETE operation internally?

A: PostgreSQL does NOT remove the tuple physically. It sets `xmax = current_xid`. After the transaction commits, the tuple becomes a dead tuple visible only to older snapshots. VACUUM later reclaims the space.

Q9. What is ctid and how does it relate to tuple versions?

A: `ctid` is a physical pointer (`page_number` , `item_index`) to the tuple's location in the heap. When a row is updated, the old tuple's `ctid` stored in the line pointer array is not changed, but the old tuple itself contains a forward pointer (also called `ctid` in the tuple header) to the new version. Index entries always point to the head of the version chain.

Q10. What is a HOT update and when does it occur?

A: A Heap-Only Tuple (HOT) update occurs when: (1) the UPDATE does not change any indexed column, and (2) the new tuple version fits on the same heap page. In this case, no new index entry is needed, reducing index

bloat and write amplification.

Q11. How does a long-running transaction affect VACUUM?

A: VACUUM can only remove dead tuples that are invisible to ALL active snapshots. A long-running transaction holds an old snapshot, preventing VACUUM from removing any dead tuple that was live when that snapshot was taken, even if more recent transactions have deleted those tuples.

Q12. What is the difference between READ COMMITTED and REPEATABLE READ in terms of snapshot acquisition?

A: READ COMMITTED takes a new snapshot at the start of each SQL statement — so it sees committed changes from other transactions that committed between statements. REPEATABLE READ takes a snapshot once at the start of the first statement and reuses it for the entire transaction — so it sees a consistent view regardless of concurrent commits.

Exercises and Solutions

Exercise 1 — Observe Tuple Versions

```
-- Setup
CREATE TABLE mvcc_test (id int PRIMARY KEY, val text);
INSERT INTO mvcc_test VALUES (1, 'v1');

-- Step 1: Check initial xmin
SELECT xmin, xmax, ctid, * FROM mvcc_test;

-- Step 2: Open a transaction, update, but do NOT commit
-- Session A:
BEGIN;
UPDATE mvcc_test SET val = 'v2' WHERE id = 1;
-- Don't commit yet

-- Step 3: From Session B, query the table
-- What do you see? Why?

-- Step 4: Commit Session A and re-query from Session B.
-- What changes?
```

Expected: Session B sees `val='v1'` (old version) until Session A commits, then with READ COMMITTED sees `val='v2'` on next query.

Exercise 2 — Dead Tuple Accumulation

```
-- Create table and update rows 1000 times
CREATE TABLE bloat_test (id int, counter int);
INSERT INTO bloat_test VALUES (1, 0);

DO $$
BEGIN
  FOR i IN 1..1000 LOOP
```

```
        UPDATE bloat_test SET counter = i WHERE id = 1;
    END LOOP;
END;
$$;

-- Check dead tuple count
SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname = 'bloat_test';

-- Run VACUUM and check again
VACUUM bloat_test;
SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname = 'bloat_test';
```

Production Troubleshooting Scenarios

Scenario 1: Table Bloat Despite Autovacuum Running

Symptom: Table is 10x larger than expected. `pg_stat_user_tables` shows autovacuum running but `n_dead_tup` stays high.

Diagnosis:

```
-- Check for long-running transactions blocking vacuum
SELECT pid, age(backend_xid) AS xid_age, state, query
FROM pg_stat_activity
WHERE backend_xid IS NOT NULL
ORDER BY xid_age DESC;

-- Check autovacuum settings for the table
SELECT relname, reloptions FROM pg_class WHERE relname = 'my_bloated_table';
```

Fix: Terminate the long-running transaction. Consider decreasing `autovacuum_vacuum_scale_factor` for this table.

Scenario 2: Snapshot Too Old Error

```
ERROR: snapshot too old
```

Cause: `old_snapshot_threshold` is set and a query is using a snapshot older than the threshold.

Fix: Shorten transactions or increase `old_snapshot_threshold`. This feature is rarely needed; consider disabling it.

Cross-References

- `01_transaction_basics.md` — Transaction lifecycle and atomicity
- `03_isolation_levels.md` — How snapshot acquisition varies by isolation level
- `07_serializable_snapshot_isolation.md` — SSI builds on MVCC snapshots
- `../10_PostgreSQL_Internals/02_heap_storage.md` — Physical layout of tuples on pages
- `../10_PostgreSQL_Internals/06_vacuum.md` — How VACUUM reclaims dead tuples

- `../10_PostgreSQL_Internals/08_visibility_map.md` — Tracks which pages have no dead tuples